

elasticsearch.

Nathalie Ling

27 février 2014

Base de données avancées 1 – NFE204

le **cnam**

## Sommaire

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 1. Présentation d'Elasticsearch .....                                    | 3  |
| 2. Notions de base.....                                                  | 4  |
| 3. Installation .....                                                    | 7  |
| 4. Les nœuds d'un cluster.....                                           | 9  |
| 5. Le cluster .....                                                      | 10 |
| 5.1. Informations sur le cluster .....                                   | 10 |
| 5.2. Paramétrage du cluster .....                                        | 11 |
| 5.3. Sauvegarde .....                                                    | 12 |
| 6. Les requêtes .....                                                    | 13 |
| 6.1. Création d'un index .....                                           | 13 |
| 6.2. Suppression d'un index .....                                        | 13 |
| 6.3. Création d'un document.....                                         | 13 |
| 6.4. Le schéma (mapping) d'un type ou d'un index .....                   | 14 |
| 6.5. Affichage d'un document .....                                       | 15 |
| 6.6. Modification d'un document.....                                     | 15 |
| 6.7. Suppression d'un document.....                                      | 16 |
| 6.8. Recherche de documents.....                                         | 16 |
| 6.9. Suppression de documents par une recherche.....                     | 17 |
| 6.10. Recherche de documents sur plusieurs index.....                    | 18 |
| 6.11. Alias sur les index .....                                          | 19 |
| 6.12. La recherche par facette (facet) .....                             | 23 |
| 7. La percolation .....                                                  | 24 |
| 8. La distribution des documents, utilisation du paramètre routing ..... | 26 |
| 9. Conclusion .....                                                      | 29 |
| Bibliographie : .....                                                    | 30 |

## 1. Présentation d'Elasticsearch

Elasticsearch est un moteur de recherche.

Il est écrit en Java et est Open source. Il utilise la librairie Lucene, écrite en Java.

Il a une base NoSQL qui contient des documents.

En 2004, Shay Banon avait développé le moteur de recherche Compass. Puis, il a réécrit une troisième version afin de gérer les montées en charge des données et le passage à l'échelle du nombre de serveurs et l'a appelé Elasticsearch. La première version d'Elasticsearch date de février 2010.

Elasticsearch est scalable à l'horizontale. On peut rajouter des serveurs au fur et à mesure des besoins. L'ajout et la suppression des nœuds se fait dynamiquement. Puis, le cluster se réorganise automatiquement. Elasticsearch peut être utilisé sur des centaines de machines.

Elasticsearch a deux fonctions :

- Il sert à insérer des documents JSON dans un index inversé. Le format des documents Elasticsearch est libre.
- Il permet de faire de la recherche plein texte dans les documents de la base et d'afficher ces documents.

Il est rapide et fiable. Il a son propre langage, le Query DSL.

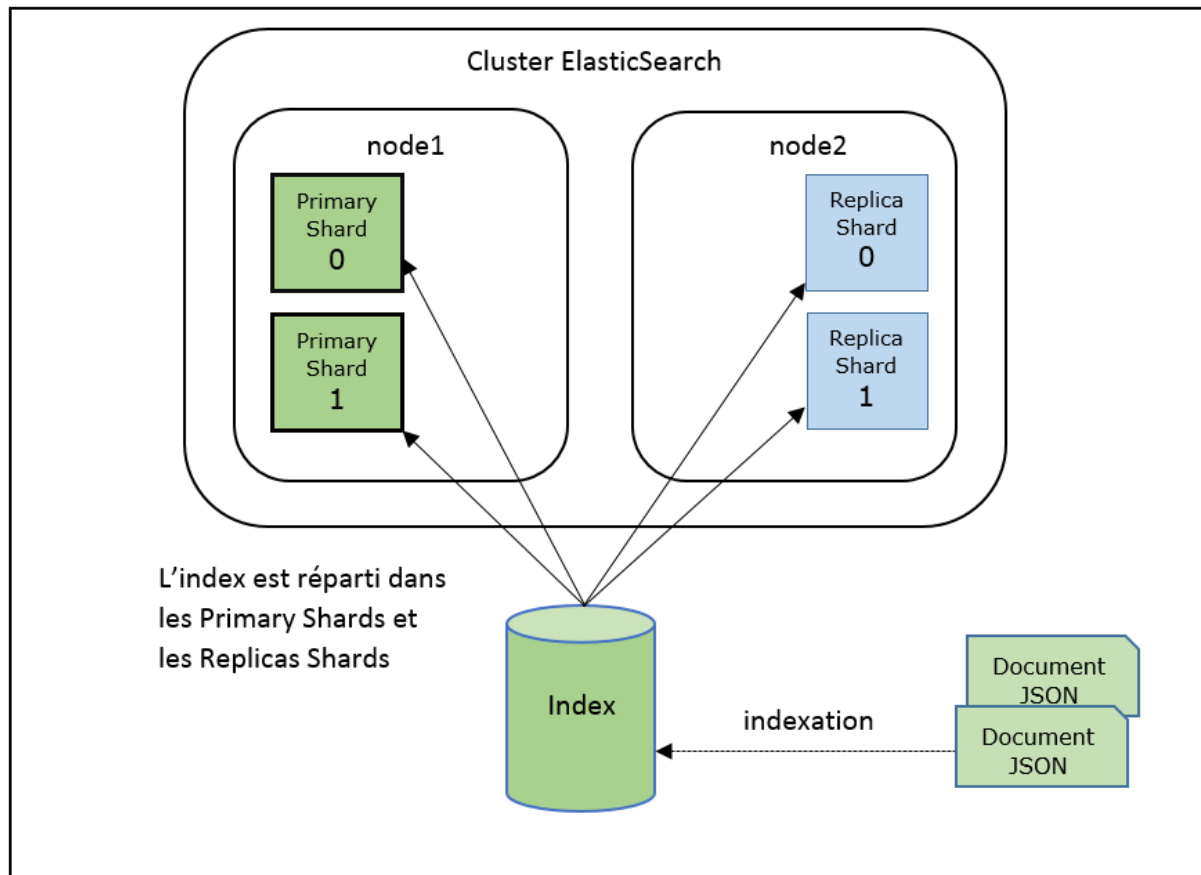
Elasticsearch s'appuie sur le protocole REST. On exécute des requêtes HTTP pour faire des opérations sur les documents ou administrer le cluster Elasticsearch.

On utilise

- POST pour une création
- PUT pour une création ou une modification
- GET pour une lecture
- DELETE pour une suppression

Le résultat des requêtes HTTP est un objet JSON.

## 2. Notions de base



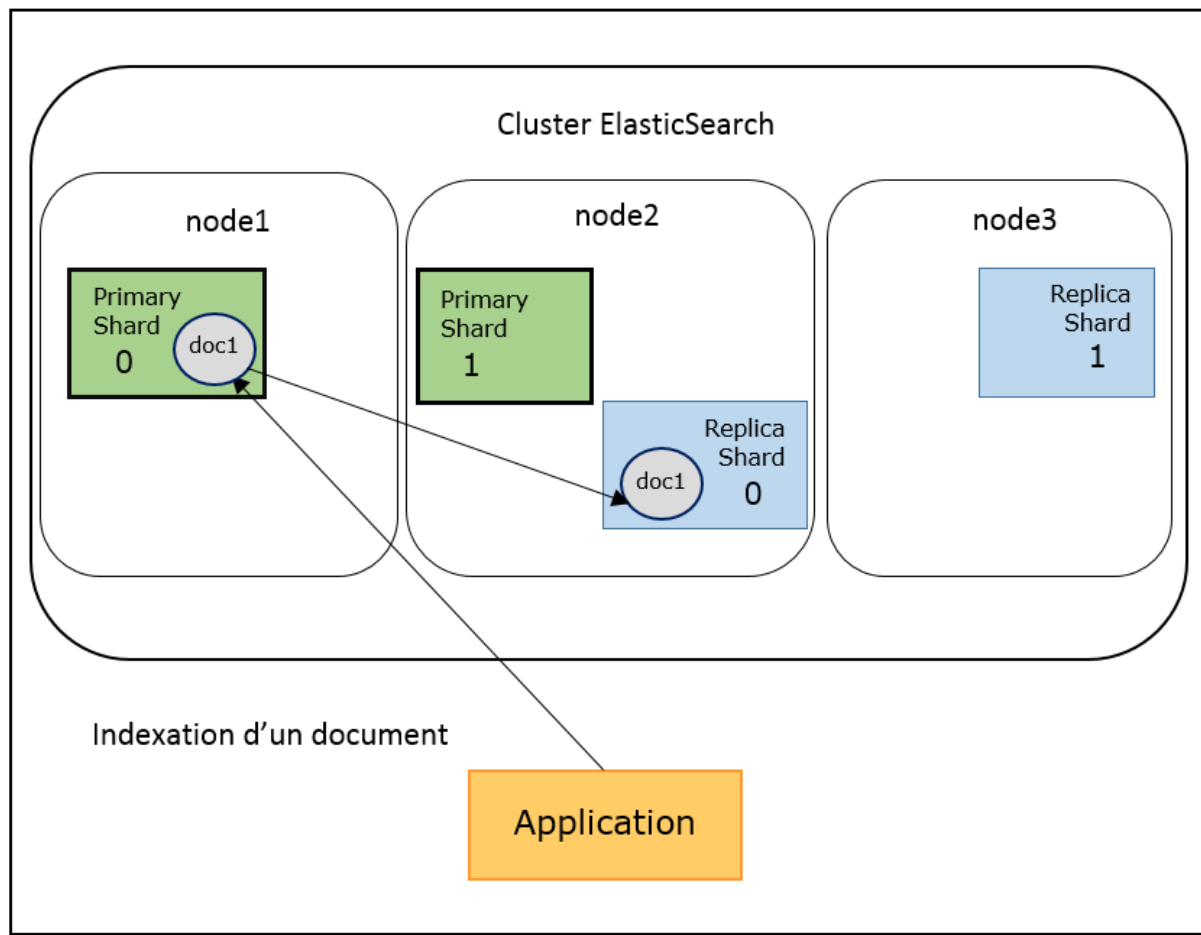
Un document JSON est inséré dans un index inversé.  
Un index inversé est une collection de documents.

Un cluster est un ensemble de nœuds.  
Un nœud (node) est une exécution d'ElasticSearch sur un serveur.

ElasticSearch est distribué.

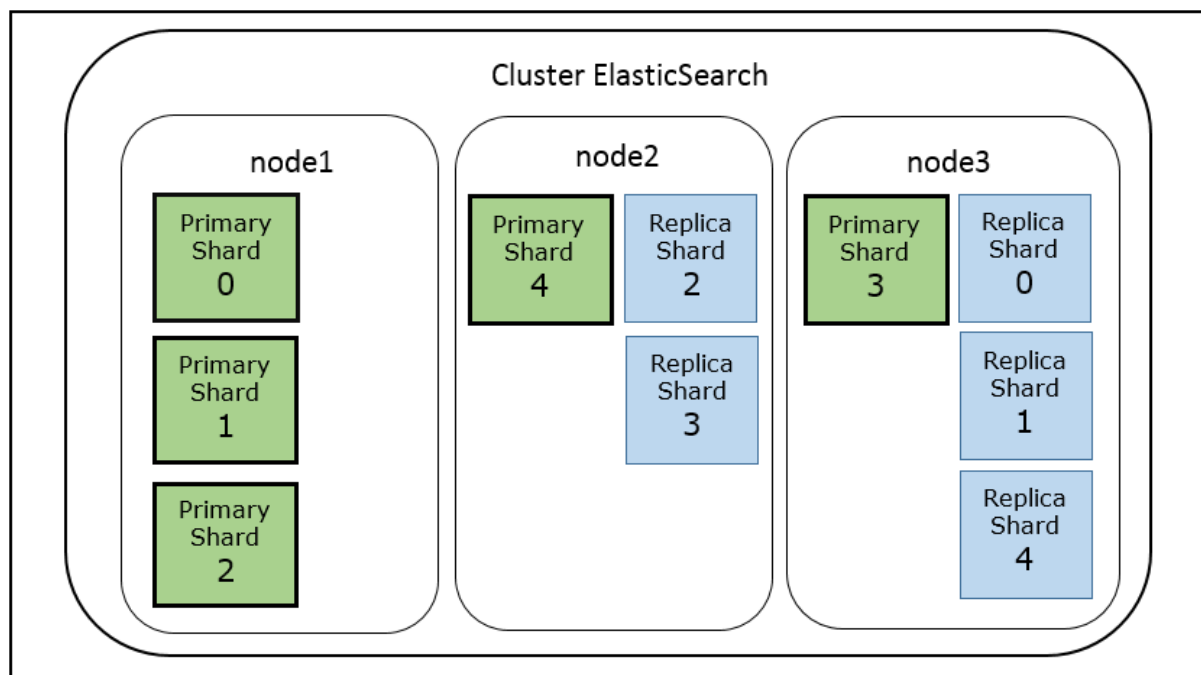
Un index est découpé en fragments (shards).  
Il y a deux types de shards : un Primary shard et un Replica shard.  
Un Replica shard est une copie d'un Primary shard.  
Les shards sont distribués dans tous les nœuds en automatique.  
Dans un nœud, on peut avoir des Primary shards et des Replica shards.

Les documents indexés sont répartis dans des Primary shards et des Replica shards.



Un document est inséré dans un Primary shard, puis est copié dans un Replica shard.

Une recherche est faite dans un Replica shard.



## La haute disponibilité

Un Primary shard a zéro ou plusieurs Replicas shards, que l'on paramètre dans le fichier `elasticsearch.yml`

|                                          |                             |
|------------------------------------------|-----------------------------|
| <code>index.number_of_shards: 5</code>   | : nombre de Primary shards  |
| <code>index.number_of_replicas: 1</code> | : nombre de Replicas shards |

Par défaut, le nombre de Primary shards est 5 et le nombre de Replicas shards est 1.

Chaque Primary shard a un Replica shard. Il y a 5 Primary shards et 5 Replicas shards.

Un Replica shard est dans un nœud différent du Primary shard auquel il correspond.

Dans l'exemple du schéma, le Primary shard 0 est sur le nœud 1.

Le Replica shard 0, la copie du Primary shard 0 est sur un nœud différent, le nœud 3.

Augmenter le nombre de Primary shards permet d'améliorer les performances lorsque l'on indexe.

Augmenter le nombre de Replica shards permet d'améliorer les performances lorsque l'on fait une recherche. Si le nombre d'utilisateurs augmente, on peut augmenter ainsi le nombre de requêtes en parallèle.

Le nombre de Replicas shards est le nombre de serveurs qui peuvent tomber en panne sans perte de données.

Dans l'exemple, on peut avoir un nœud qui tombe en panne et toujours avoir tous les index en ligne.

Quand un Primary shard tombe et ne fait plus partie du cluster, son Replica shard est promu Primary shard.

### 3. Installation

Installation de 3 noeuds Elasticsearch version 0.90.9 sur un pc en Windows 8.

Prérequis : avoir installé Java Runtime Environment (JRE 6 minimum).

Positionner la variable système JAVA\_HOME.

Installation du premier nœud :

Dézipper le fichier elasticsearch-0.90.9.zip sur C:\ .

Renommer le répertoire elasticsearch-0.90.9 en elasticsearch\_1 .

Modifier le nom du cluster et le nom du nœud dans le fichier elasticsearch.yml  
situé dans le répertoire config

```
cluster.name: moncluster
node.name: "lenoeud1"
```

Pour installer un second nœud :

Faire une copie du répertoire elasticsearch\_1 et le renommer en elasticsearch\_2 .

Supprimer les répertoires data et logs.

Modifier le fichier elasticsearch.yml

```
node.name: "lenoeud2"
```

Pour installer un troisième nœud :

Copier le répertoire c:\elasticsearch\_2 et le renommer c:\elasticsearch\_3 .

Modifier le fichier elasticsearch.yml

```
node.name: "lenoeud3"
```

Pour exécuter les instances Elasticsearch des trois nœuds, double-clic sur

C:\elasticsearch\_1\bin\elasticsearch.bat

C:\elasticsearch\_2\bin\elasticsearch.bat

C:\elasticsearch\_3\bin\elasticsearch.bat

Dans un navigateur, on saisit l'URL <http://localhost:9200> , on obtient

```
{
  "ok" : true,
  "status" : 200,
  "name" : "lenoeud1",
  "version" : {
    "number" : "0.90.9",
    "build_hash" :
"a968646da4b6a2d9d8bca9e51e92597fe64e8d1a",
    "build_timestamp" : "2013-12-23T10:35:28Z",
    "build_snapshot" : false,
    "lucene_version" : "4.6"
  },
  "tagline" : "You Know, for Search"
}
```

: nom du nœud 1

: version d'Elasticsearch

: date d'installation du nœud 1

: version de Lucene

Installer Cygwin pour exécuter des commandes en utilisant curl.

Installer le plug-in head pour avoir une interface graphique du cluster et exécuter des requêtes.

Arrêter le cluster Elasticsearch

- par la commande

```
curl -XPOST 'http://localhost:9200/_shutdown'
```

- ou fermer les fenêtres Elasticsearch avec

```
<CTRL>+<C>
```

Terminer le programme de commandes (O/N) ? saisir O

Ouvrir une fenêtre Invite de commandes

```
cd C:\elasticsearch_1\bin  
plugin -install mobz/elasticsearch-head
```

Exécuter les instances d'Elasticsearch des trois nœuds.

Pour utiliser le plug-in head, saisir l'URL [http://localhost:9200/\\_plugin/head/](http://localhost:9200/_plugin/head/) dans un navigateur.

Dans l'onglet Aperçu

- on visualise l'état du cluster, les nœuds du cluster, les index, la répartition des shards de chaque index dans les différents nœuds et les alias sur les index.
- On peut avoir des informations sur le cluster, les nœuds, des statistiques sur les nœuds, les index et le nombre de documents dans un index.
- On peut arrêter un nœud et faire des actions sur un index, comme par exemple le créer ou le supprimer.

L'onglet Navigateur permet

- de visualiser tous les index, tous les types de documents, tous les champs des documents et tous les documents.
- de faire une sélection d'un ou plusieurs index, ou de types de documents et d'afficher les documents de la sélection. De trier la liste des documents suivant un champ.

L'onglet Requêtes structurées permet de créer des requêtes avec une aide sur les critères de recherche et d'afficher la requête générée en Query DSL et les documents trouvés. L'affichage du résultat est sous forme d'un tableau de documents ou en JSON ou en format CSV.

L'onglet Autres requêtes permet d'exécuter des requêtes HTTP.



## 4. Les nœuds d'un cluster

Un nœud peut avoir 3 rôles :

- Le **nœud maître** (Master node) gère l'état du cluster et distribue les shards dans tous les nœuds.  
Dans un cluster, un seul nœud est élu nœud maître.  
Quand il tombe en panne, le cluster fait l'élection d'un autre nœud maître automatiquement parmi tous les nœuds qui ont un rôle de nœud maître.  
Le nœud maître détecte et supprime les nœuds qui ne sont plus en ligne. Puis réorganise les shards dans les nœuds automatiquement.

Paramétrage du fichier `elasticsearch.yml` pour un nœud qui peut être élu nœud maître :

```
node.master: true  
node.data: true
```

ou

```
node.master: true  
node.data: false
```

Le nœud élu nœud maître est indiqué dans le fichier log, `nom_du_cluster.log`, par exemple `moncluster.log`

```
[2014-01-27 12:38:04,797][INFO ][cluster.service      ] [lenoeud1]  
new_master [lenoeud1][3ZcKKhM1S7mS3wfypv8fXA]  
[inet[/10.1.56.222:9300]], reason: zen-disco-join (elected_as_master)
```

- Le **nœud de données** (Data node). Il contient les shards.

Dans le fichier `elasticsearch.yml`, on a

```
node.master: true  
node.data: true
```

ou

```
node.master: false  
node.data: true
```

- Le **nœud non Master et non Data**. Toutes les requêtes HTTP qui sont envoyées au cluster passent par ce nœud.  
Il fait de la répartition de charge (load balancing), il route les requêtes vers les nœuds de données et regroupe les résultats d'une requête trouvés dans chaque shard.

Dans le fichier `elasticsearch.yml`, on a

```
node.master: false  
node.data: false
```

## Le mode découverte (Zen discovery)

Lorsque l'on démarre un nouveau nœud, Elasticsearch cherche s'il y a un nœud maître avec le même nom de cluster. S'il y en a un, le nouveau nœud rejoint le cluster. S'il n'y a pas de nœud maître, le nœud devient nœud maître.

Le nœud fait des ping en mode multicast ou unicast pour chercher les autres nœuds.

- En mode multicast, le nœud envoie un message à tous les autres nœuds. Seuls, les nœuds du même cluster répondent.
- Le mode unicast est conseillé pour les clusters utilisés en production. Le nœud se connecte à un autre nœud du cluster que l'on a listé dans le fichier de configuration `elasticsearch.yml`.

```
discovery.zen.ping.multicast.enabled: false
discovery.zen.ping.unicast.hosts: ["host1", "host2:9300",
"host3[9300-9399]"]
```

## 5. Le cluster

### 5.1. Informations sur le cluster

Commande qui affiche des informations sur le cluster

```
curl localhost:9200/_cluster/health?pretty
```

```
{
  "cluster_name" : "moncluster",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 3,
  "number_of_data_nodes" : 3,
  "active_primary_shards" : 51,
  "active_shards" : 103,
  "relocating_shards" : 0,

  "initializing_shards" : 0,
  "unassigned_shards" : 0
}
```

|                                                                         |
|-------------------------------------------------------------------------|
| : nom du cluster                                                        |
| : état du cluster                                                       |
| : nombre de nœuds dans le cluster                                       |
| : nombre de nœuds de données                                            |
| : nombre de Primary shards actifs                                       |
| : nombre de shards actifs                                               |
| : nombre de shards en cours de déplacement d'un nœud vers un autre nœud |
| : nombre de shards en cours d'initialisation                            |
| : nombre de shards qui ne sont pas affectés à un nœud                   |

## L'état du cluster

- `status : red`  
Un index a un shard qui ne soit pas dans le cluster.  
unassigned\_shards indique le nombre de shards concernés.
- `status : yellow`  
Le cluster est opérationnel, on peut indexer des documents et faire des recherches. L'index est distribué dans les Primary shards. Mais, les Replica shards ne sont pas affectés à un nœud.
- `status : green`  
Le cluster est complètement opérationnel. Les Primary shards et les Replica shards appartiennent à des nœuds.

Un shard peut ne pas être affecté à un nœud (unassigned), il attend qu'un nœud soit démarré pour faire partie du cluster.

Par exemple, le nombre de shards d'un index est 5.  
On paramètre le nombre maximum de shards dans un nœud pour un index à 4.  
Avec la commande suivante, il y a 4 shards maximum dans un nœud pour l'index magasin

```
curl -XPUT 'localhost:9200/magasin/_settings' -d '{  
  "index.routing.allocation.total_shards_per_node" : 4}'
```

Après avoir démarré un seul nœud, il y a un shard qui n'est pas affecté au nœud. Le shard attend qu'un second nœud soit démarré pour en faire partie.

## 5.2. Paramétrage du cluster

Pour éviter des rééquilibrages temporaires des shards dans les nœuds au démarrage du cluster, lorsque les nœuds ne sont pas encore tous en ligne, on peut définir trois paramètres dans le fichier elasticsearch.yml

|                                 |                                                                                                                                                                                                                                                                                         |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gateway.recover_after_nodes : 1 | Nombre de nœuds minimum pour démarrer le cluster.                                                                                                                                                                                                                                       |
| gateway.recover_after_time : 5m | Après que le nœud ait démarré, le cluster attend 5 minutes que tous les nœuds attendus soient en ligne pour commencer l'initialisation des shards (recover). Lorsque tous les nœuds sont en ligne, ce paramètre est ignoré et le cluster effectue aussitôt la restauration des données. |
| gateway.expected_nodes : 3      | Nombre de nœuds attendus, dans l'exemple, on a 3 nœuds dans le cluster                                                                                                                                                                                                                  |

Si un ou plusieurs nœuds ont une perte de connexion réseau, et ne voient plus le nœud maître, un des ces nœuds peut être élu nœud maître et ces nœuds peuvent créer leur propre cluster. On se retrouve avec deux clusters avec chacun un nœud maître. Pour éviter le split brain, on paramètre dans le fichier `elasticsearch.yml`

|                                                    |                                                                                                   |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>discovery.zen.minimum_master_nodes: 3</code> | Par exemple pour un cluster avec 5 nodes, on positionne le <code>minimum_master_nodes</code> à 3. |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------|

Ce paramètre définit le nombre minimum de nœuds présents, qui peuvent être élus nœud maître, pour qu'il y en ait un qui soit élu nœud maître et former un cluster.

Ce nombre est idéalement paramétré à 50% du nombre de nœuds éligibles nœud maître +1.

### 5.3. Sauvegarde

Avant d'être exécutées, les transactions à effectuer sur les documents sont écrites dans un fichier translog.

La commande `flush` sur un index libère la mémoire utilisée par l'index, les transactions du fichier translog sont exécutées et le fichier translog est réinitialisé.

Sauvegarde des index avec la version 0.90.9 d'Elasticsearch (parue le 23 décembre 2013) :

- Sauvegarde à froid (avec arrêt du cluster):  
arrêter le cluster et copier le répertoire data de tous les nœuds.
- Sauvegarde à chaud (sans arrêter le cluster)
  1. Désactiver le flush des fichiers translog

```
curl -XPUT 'localhost:9200/_settings' -d
  '{ "index" : { "translog.disable_flush" : "true" } }'
```

2. Copier les répertoires data de chaque nœud (ou du nœud s'il a tous les shards des index)

3. Réactiver le flush des fichiers translog.

```
curl -XPUT 'localhost:9200/_settings' -d
  '{ "index" : { "translog.disable_flush" : "false" } }'
```

A partir de la version 1.0.0 (parue le 12 février 2014), on peut faire une sauvegarde en incrémental (snapshot) et une restauration des index ou du cluster.

La dernière version d'Elasticsearch à ce jour, est la 1.0.1 est sortie le 25 février 2014.

## 6. Les requêtes

### 6.1. Création d'un index

Création de l'index biblio2

```
curl -XPUT 'http://localhost:9200/biblio2'
```

### 6.2. Suppression d'un index

Suppression de l'index biblio2

```
curl -XDELETE 'http://localhost:9200/biblio2'
```

### 6.3. Création d'un document

Lorsque l'on crée un document, l'index est créé automatiquement s'il n'existe pas.

Pour la création d'un document avec un id, on indique une URI sous la forme `http://serveur:port/index/type/id`

Création du document **cd1** de type **cd** dans la collection **biblio** et création de la collection **biblio** qui n'existe pas

```
curl -XPUT 'http://localhost:9200/biblio/cd/cd1' -d '{
    "titre": "Lohengrin",
    "compositeur": "Richard Wagner"
}'

{"ok":true,"_index":"biblio","_type":"cd","_id":"cd1","_version":1}
```

Un document Elasticsearch a un type, un id, une version.

- L'identifiant du document :  
Si on n'indique pas un id à la création d'un document, Elasticsearch en crée un automatiquement.

Création d'un document de type **cd** dans la collection **biblio**

```
curl -XPOST 'http://localhost:9200/biblio/cd' -d '{
    "titre": "Don Giovanni",
    "compositeur": "Wolfgang Amadeus Mozart"
}'

{"ok":true,"_index":"biblio","_type":"cd",
"_id":"ZcfvFnEcQLeLpYUKRmbL2w","_version":1}
```

Elasticsearch a créé l'id `ZcfvFnEcQLeLpYUKRmbL2w` pour le document.

- La version d'un document sert à la concurrence d'accès, pour qu'il n'y ait pas de mise à jour en même temps qu'une lecture.  
Par défaut, le numéro d'une version d'un document commence à 1 et puis est incrémenté de 1 à chaque mise à jour.  
On peut spécifier la version d'un document quand on crée ou quand on modifie un document. La valeur doit être numérique, comprise entre 1 et  $(9,2 \times 10^{18})$  et doit être supérieure ou égale à la version du document déjà indexé, sinon, le document n'est pas indexé.
- Un type de document est décrit par un schéma.  
On peut avoir plusieurs types de document dans un index.

#### 6.4. Le schéma (mapping) d'un type ou d'un index

Le schéma est une métadonnée. Il n'est pas nécessaire de le créer. Il a été détecté automatiquement par Elasticsearch à la création du document cd1. Il peut être aussi créé manuellement auparavant.

Le schéma du type **cd** de la collection **biblio**

```
curl -XGET 'http://localhost:9200/biblio/cd/_mapping'
  {"cd" : {
    "properties" : {
      "compositeur" : {"type" : "string"},
      "titre" : {"type" : "string"}
    }
  }
}
```

L'option **?pretty=true** permet d'avoir un affichage plus lisible.

Le schéma de l'index biblio

```
curl -XGET 'http://localhost:9200/biblio/_mapping'?pretty=true
```

|                                                                                                                                                                                                                |                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>{   "biblio" : {     "cd" : {       "properties" : {         "compositeur" : {           "type" : "string"         },         "titre" : {           "type" : "string"         }       }     }   } }</pre> | <p>: nom de l'index</p> <p>: nom du type de document</p> <p>: champ compositeur</p> <p>: type du champ compositeur</p> <p>: champ titre</p> <p>: type du champ titre</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6.5. Affichage d'un document

Affichage du document **cd1** de type **cd** dans la collection **biblio**

```
curl -XGET 'http://localhost:9200/biblio/cd/cd1'

{"_index":"biblio","_type":"cd","_id":"cd1","_version":1,"exists":true,
 "_source":
  {"titre": "Lohengrin",
   "compositeur": "Richard Wagner"}
```

Le document original est dans **\_source** .

On peut l'afficher avec la commande

```
curl -XGET 'http://localhost:9200/biblio/cd/cd1/_source'
```

## 6.6. Modification d'un document

Création du document **cd3** de type **cd** dans la collection **biblio** puis mise à jour du champ **compositeur**

```
curl -XPOST 'http://localhost:9200/biblio/cd/cd3' -d '{
  "titre": "Le vaisseau fantôme",
  "compositeur": "Richard Wagner"
}'

{"ok":true,"_index":"biblio","_type":"cd","_id":"cd3","_version":1}
```

```
curl -XPUT 'http://localhost:9200/biblio/cd/cd3' -d '{
  "titre": " Le vaisseau fantôme ",
  "compositeur": "Wilhelm Richard Wagner"
}'

{"ok":true,"_index":"biblio","_type":"cd","_id":"cd3","_version":2}
```

La version du document est 2.

## 6.7. Suppression d'un document

Suppression du document précédent

```
curl -XDELETE 'http://localhost:9200/biblio/cd/cd3'

{"ok":true,"found":true,"_index":"biblio","_type":"cd","_id":"cd3",
 "_version":3}
```

On a une nouvelle version du document qui est 3.

Le document est seulement marqué comme supprimé. Il sera réellement supprimé de la base après fusion (merge) des segments Lucene.

Pour cela, la commande **optimize** peut être lancée manuellement, elle fusionne et réduit le nombre de segments Lucene contenu dans un shard.

On exécute la commande sur l'index biblio

```
curl -XPOST 'http://localhost:9200/biblio/_optimize'
```

La commande optimize est habituellement utilisée pour optimiser un ou plusieurs index, afin d'accélérer les recherches.

## 6.8. Recherche de documents

Les requêtes Query DSL sont sous la forme :

```
{
  "query" {
    // la requête DSL
  }
}
```

Recherche d'un document de type **cd** dans la collection **biblio** qui a **wagner** pour **compositeur**

```
curl -XGET 'http://localhost:9200/biblio/cd/_search' -d '{
  "query" : {
    "text" : { "compositeur" : "wagner" }
  }
}'
```



Ou avec une requête simple

```
curl -XGET 'http://localhost:9200/biblio/cd/_search?q=compositeur:wagner
&pretty=true'
```

```
{
  "took" : 6,
  "timed_out" : false,
  "_shards" : {
    "total" : 5,
    "successful" : 5,
    "failed" : 0
  },
  "hits" : {
    "total" : 1,
    "max_score" : 0.625,
    "hits" : [ {
      "_index" : "biblio",
      "_type" : "cd",
      "_id" : "cd1",
      "_score" : 0.625,
      "_source" : {
        "titre" : "Lohengrin",
        "compositeur" : "Richard Wagner"
      }
    } ]
  }
}
```

] Le nombre de documents trouvés

] Le document trouvé est /biblio/cd/cd1

] Le document source

Le résultat d'une requête est une liste de documents triée par défaut par le score du document.

Le score est calculé. Il indique la pertinence du document par rapport à la requête.

## 6.9. Suppression de documents par une recherche

On cherche les documents de type **cd** dans la collection **biblio** qui contiennent **giovanni** dans le **titre** et on les supprime.

```
curl -XDELETE 'http://localhost:9200/biblio/cd/_query?q=titre:giovanni'
```

Le document /biblio/cd/ZcfvFnEcQLeLpYUKRmbL2w est supprimé.

## 6.10. Recherche de documents sur plusieurs index

Création du document **dvd1** de type **dvd** dans la collection **mediatheque** et création de la collection **mediatheque** qui n'existe pas

```
curl -XPOST 'http://localhost:9200/mediatheque/dvd/dvd1' -d '{
  "titre": "Le vaisseau fantôme",
  "compositeur": "Richard Wagner"}'
```

Recherche de **wagner** dans tous les champs de n'importe quel document dans tous les index.

On trouve deux documents : /biblio/cd/cd1 et /mediatheque/dvd/dvd1 .

```
curl -XGET 'http://localhost:9200/_search?q=wagner&pretty=true'

...  "hits" : {
    "total" : 2,
    "max_score" : 0.5,
    "hits" : [ {
      "_index" : "biblio",
      "_type" : "cd",
      "_id" : "cd1",
      "_score" : 0.5,
      "_source" : {
        "titre": "Lohengrin",
        "compositeur": "Richard Wagner"
      }
    }, {
      "_index" : "mediatheque",
      "_type" : "dvd",
      "_id" : "dvd1",
      "_score" : 0.13424811,
      "_source" : {
        "titre": "Le vaisseau fantôme",
        "compositeur": "Richard Wagner"
      }
    } ]
  }
}
```

Recherche de **wagner** dans le champ **compositeur** d'un document dans les index **biblio** et **mediatheque**.

On trouve les mêmes documents /biblio/cd/cd1 et /mediatheque/dvd/dvd1 avec des scores différents.

```
curl -XGET
'http://localhost:9200/biblio,mediatheque/_search?q=compositeur:
wagner&pretty=true'

... "hits" : {
  "total" : 2,
  "max_score" : 0.625,
  "hits" : [ {
    "_index" : "biblio",
    "_type" : "cd",
    "_id" : "cd1",
    "_score" : 0.625,
    "_source" : {
      "titre" : "Lohengrin",
      "compositeur" : "Richard Wagner"
    }
  }, {
    "_index" : "mediatheque",
    "_type" : "dvd",
    "_id" : "dvd1",
    "_score" : 0.19178301,
    "_source" : {
      "titre" : "Le vaisseau fantôme",
      "compositeur" : "Richard Wagner"
    }
  } ]
}
```

### 6.11. Alias sur les index

On donne un autre nom à un index ou plusieurs index.

Création d'un alias sur l'index mediatheque.

```
curl -XPOST 'http://localhost:9200/_aliases' -d '
{
  "actions" : [
    { "add" : { "index" : "mediatheque", "alias" : "alias1" } }
  ]
}'
```

Création d'un alias sur 2 index mediatheque et biblio

```
curl -XPOST 'http://localhost:9200/_aliases' -d '{
  "actions" : [
    { "add" : { "index" : "mediatheque", "alias" : "alias2" } },
    { "add" : { "index" : "biblio", "alias" : "alias2" } }
  ]
}'
```

On peut faire des requêtes sur un alias à l'identique d'un index.

Recherche de **wagner** dans le champ **compositeur** sur **alias2**.

On trouve les mêmes documents /biblio/cd/cd1 et /mediatheque/dvd/dvd1 avec les mêmes scores que la requête précédente sans alias sur les index mediatheque et biblio.

```
curl -XGET
'http://localhost:9200/alias2/_search?q=compositeur:wagner&pretty=true'

... "hits" : {
  "total" : 2,
  "max_score" : 0.625,
  "hits" : [ {
    "_index" : "biblio",
    "_type" : "cd",
    "_id" : "cd1",
    "_score" : 0.625,
    "_source" : {
      "titre": "Lohengrin",
      "compositeur": "Richard Wagner"
    }
  }, {
    "_index" : "mediatheque",
    "_type" : "dvd",
    "_id" : "dvd1",
    "_score" : 0.19178301,
    "_source" : {
      "titre": "Le vaisseau fantôme",
      "compositeur": "Richard Wagner"
    }
  } ]
}
```

Un filtre permet d'améliorer les résultats des requêtes.

Création d'un alias **aliaswagner** sur 2 index **mediatheque** et **biblio** avec un filtre **wagner** sur le champ **compositeur**

```
curl -XPOST 'http://localhost:9200/_aliases' -d '{
  "actions" : [
    {"add" : {"index" : "mediatheque", "alias" : "aliaswagner",
              "filter" : { "term" : { "compositeur" : "wagner" }}}},
    {"add" : {"index" : "biblio", "alias" : "aliaswagner",
              "filter" : { "term" : { "compositeur" : "wagner" }}}}}
  ]
}'

{"ok":true,"acknowledged":true}
```

Recherche de document avec **le** dans le champ **titre** dans les index **mediatheque** et **biblio** avec un filtre sur le compositeur **wagner** .  
On trouve le document /mediatheque/dvd/dvd1 .

```
curl -XGET
'http://localhost:9200/aliaswagner/_search?q=titre:le&pretty=true'

... "hits" : {
  "total" : 1,
  "max_score" : 0.15342641,
  "hits" : [ {
    "_index" : "mediatheque",
    "_type" : "dvd",
    "_id" : "dvd1",
    "_score" : 0.15342641,
    "_source" : {
      "titre": "Le vaisseau fantôme",
      "compositeur": "Richard Wagner"
    }
  } ]
}
```

Visualiser tous les alias du cluster

```
curl -XGET 'http://localhost:9200/_aliases'
```

Visualiser les alias pour les 2 index mediatheque et biblio.

L'index biblio a deux alias : alias2 et aliaswagner.

L'index mediatheque a trois alias : alias2, alias1 et aliaswagner.

```
curl -XGET
'http://localhost:9200/mediatheque,biblio/_aliases?pretty=true'

{
  "biblio" : {
    "aliases" : {
      "alias2" : { },
      "aliaswagner" : {
        "filter" : {
          "term" : {
            "compositeur" : "wagner"
          }
        }
      }
    }
  },
  "mediatheque" : {
    "aliases" : {
      "alias2" : { },
      "alias1" : { },
      "aliaswagner" : {
        "filter" : {
          "term" : {
            "compositeur" : "wagner"
          }
        }
      }
    }
  }
}
```

Suppression de l'alias aliaswagner

```
curl -XPOST 'http://localhost:9200/_aliases' -d '
{
  "actions" : [
    {"remove" : {"index" : "mediatheque", "alias" : "aliaswagner"}},
    {"remove" : {"index" : "biblio", "alias" : "aliaswagner"}}
  ]
}'

{"ok":true,"acknowledged":true}
```

## 6.12. La recherche par facette (facet)

Création du document **cd4** de type **cd** dans la collection **biblio**

```
curl -XPOST 'http://localhost:9200/biblio/cd/cd4' -d '{
  "titre": "Tristan et Iseult",
  "compositeur": "Richard Wagner"
}'

{"ok":true,"_index":"biblio","_type":"cd","_id":"cd4","_version":1}
```

La recherche par facette permet de faire des agrégats sur les résultats de recherche.

On fait une recherche par facette dans les documents de type **cd** dans **biblio**.

La facette est le champ **compositeur**.

On trouve trois documents /biblio/cd/cd4 /biblio/cd/cd2 et /biblio/cd/cd1 , avec deux documents du compositeur Richard Wagner et un document du compositeur Gioacchino ROSSINI.

```
curl -X POST "http://localhost:9200/biblio/cd/_search?pretty=true" -d '
{
  "facets" : {
    "tags" : { "terms" : { "script_field" : "_source.compositeur" } }
  }
}'

{
... "hits" : {
  "total" : 3,
  "max_score" : 1.0,
  "hits" : [ {
    "_index" : "biblio",
    "_type" : "cd",
    "_id" : "cd4",
    "_score" : 1.0,
    "_source" : {
      "titre": "Tristan et Iseult",
      "compositeur": "Richard Wagner"
    }
  }, {
    "_index" : "biblio",
    "_type" : "cd",
    "_id" : "cd2",
    "_score" : 1.0,
    "_source" : {
      "titre": "Le barbier de Séville",
      "compositeur": "Gioacchino ROSSINI"
    }
  }
}
```

```

    }, {
      "_index" : "biblio",
      "_type" : "cd",
      "_id" : "cd1",
      "_score" : 1.0,
      "_source" : {
        "titre": "Lohengrin",
        "compositeur": "Richard Wagner"
      }
    } ]
  },
  "facets" : {
    "tags" : {
      "_type" : "terms",
      "missing" : 0,
      "total" : 3,
      "other" : 0,
      "terms" : [ {
        "term" : "Richard Wagner",
        "count" : 2
      }, {
        "term" : "Gioacchino ROSSINI",
        "count" : 1
      } ]
    }
  }
}

```

Pour améliorer les recherches dans les documents, les champs des documents peuvent être analysés lorsqu'ils sont insérés dans la base.

On crée un index avec un analyseur (analyzer) et en option un tokenizer (pour découper le texte contenu dans les documents en mots) avec des filtres qui modifient ou suppriment des mots. Exemples de filtres pour réduire le nombre de mots, indication que la langue utilisée dans les documents, supprimer les caractères non ASCII (ASCII folding filter), faire du stemming pour prendre la racine des mots, stopwords pour supprimer des mots non significatifs.

On peut également utiliser un analyseur dans les requêtes.

## 7. La percolation

La percolation est l'opération inverse de ce que fait Elasticsearch habituellement, au lieu de d'indexer des documents et de faire des requêtes pour afficher les documents. On enregistre des requêtes percolateurs dans un index, puis on envoie ces requêtes avec un document, en retour on a les requêtes percolateurs qui correspondent au document.

Les documents ne sont pas insérés dans la base.



Fonction : Cela permet de trouver un document qui ne sera pas affiché par une requête percolateur. Puis, de créer une requête non percolateur pour afficher le document ou supprimer le document.

Dans la version 0.90.9 d'Elasticsearch, les requêtes sont dans un index spécifique `_percolator`. Cet index est répliqué sur tous les nœuds.

On crée un index **myindex**

```
curl -XPUT 'http://localhost:9200/myindex'

{"ok":true,"acknowledged":true}
```

On crée une requête percolateur. On cherche un document dans la collection **myindex** qui a **wagner** dans le champ **compositeur**.

```
curl -XPUT localhost:9200/_percolator/myindex/requetewagner -d '{
  "query" : {
    "text" : {
      "compositeur" : "wagner"}
    }
  }'

{"ok":true,"_index":"_percolator","_type":"myindex",
 "_id":"requetewagner","_version":1}
```

On crée une autre requête percolateur. On cherche un document dans la collection **myindex** qui a **rossini** dans le champ **compositeur**.

```
curl -XPUT 'localhost:9200/_percolator/myindex/requeterossini' -d '{
  "query" : {
    "text" : {
      "compositeur" : "rossini"}
    }
  }'

{"ok":true,"_index":"_percolator","_type":"myindex",
 "_id":"requeterossini","_version":1}
```

On envoie au percolateur un document avec **Richard Wagner** dans le champ **compositeur**, cela retourne la requête percolateur requetewagner.

```
curl -XGET 'localhost:9200/myindex/cd/_percolate' -d '{
  "doc" : {
    "titre": "Lohengrin",
    "compositeur": "Richard Wagner"
  }
}'

{"ok":true,"matches":["requetewagner"]}
```

On envoie au percolateur un autre document avec **Gioacchino ROSSINI** dans le champ **compositeur**, cela retourne la requête percolateur requeterossini.

```
curl -XGET 'localhost:9200/myindex/cd/_percolate' -d '{
  "doc" : {
    "titre": "Le barbier de Séville",
    "compositeur": "Gioacchino ROSSINI"
  }
}'

{"ok":true,"matches":["requeterossini"]}
```

On envoie au percolateur un troisième document avec **Emir Kusturica** dans le champ **compositeur**, cela ne retourne pas de requête.

```
curl -XGET 'localhost:9200/myindex/cd/_percolate' -d '{
  "doc" : {
    "titre": "Le temps des gitans",
    "compositeur": "Emir Kusturica"
  }
}'

{"ok":true,"matches":[""]}
```

## 8. La distribution des documents, utilisation du paramètre routing

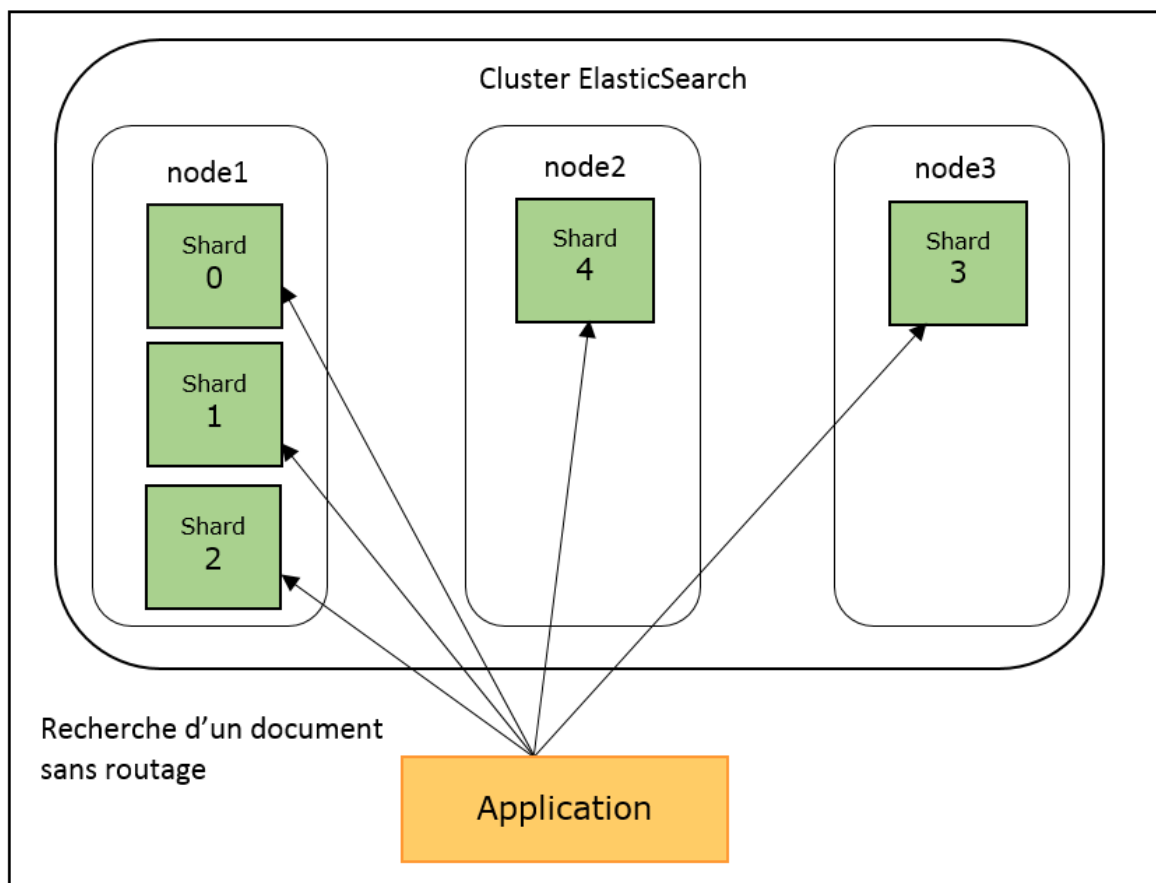
Lorsque l'on indexe un document sans préciser de paramètre de routage, Elasticsearch calcule une clé de hachage avec l'id du document, puis route le document vers un shard par rapport à la clé de hachage.

Lorsque l'on indexe un document en précisant une valeur de routage (routing), Elasticsearch calcule une clé de hachage avec cette valeur, puis route le document par rapport à la clé de hachage. Cette valeur est par exemple un mot, un nombre, un nom d'utilisateur ou une date.

On insère un document **cd2** de type **cd** dans la collection **biblio** avec **opera** comme valeur de routage.

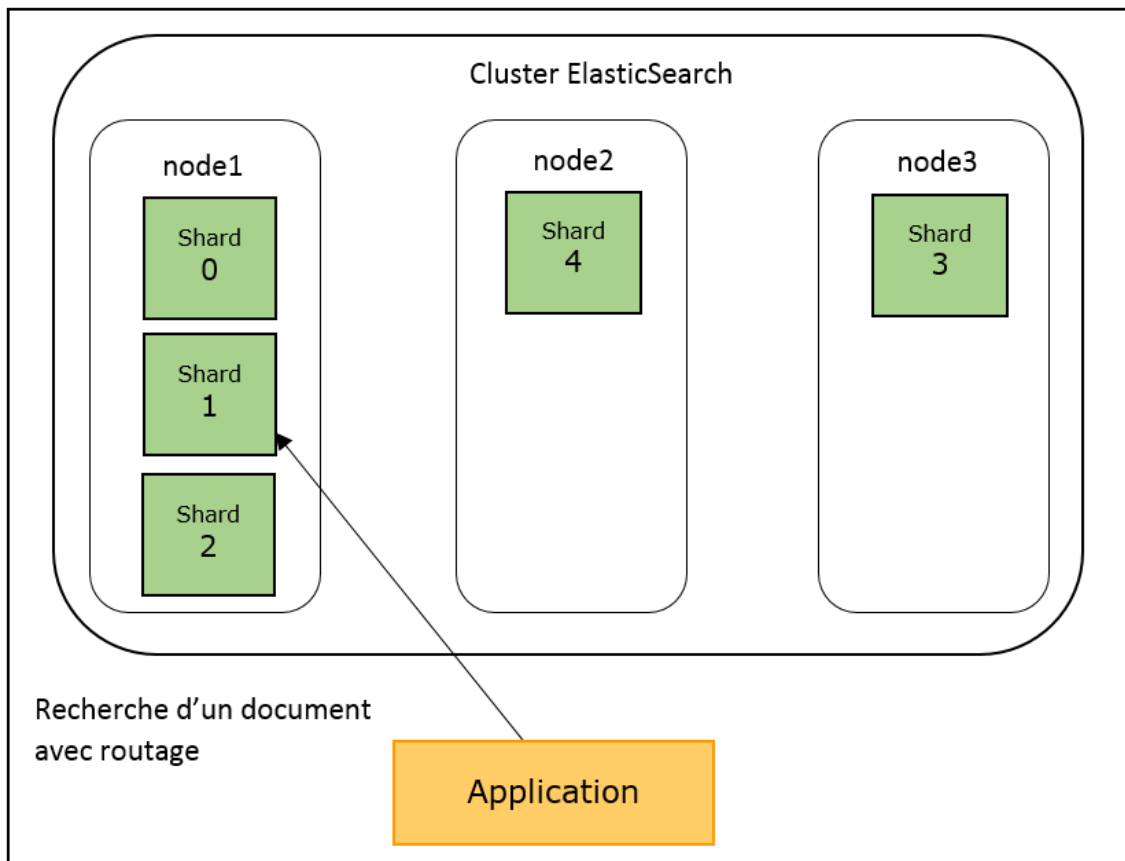
```
curl -XPUT 'http://localhost:9200/biblio/cd/cd2?routing=opera' -d '{
  "titre": "Le barbier de Séville",
  "compositeur": "Gioacchino ROSSINI"
}'
```

```
{"ok":true,"_index":"biblio","_type":"cd","_id":"cd2","_version":1}
```



Sans routage, la recherche se fait sur tous les shards.

Chaque nœud de données exécute des opérations de recherche au niveau des shards, puis rassemble les résultats trouvés.



En faisant une recherche en précisant une valeur dans routing, la recherche est beaucoup plus rapide, car les données seront cherchées dans un seul shard au lieu de tous les shards, et cela consomme moins de CPU. Dans l'exemple du schéma, seul le nœud 1 travaille.

On peut prendre la valeur de routage dans le document, on le spécifie dans le schéma de l'index.

Création d'un index magasin

```
curl -XPUT 'http://localhost:9200/magasin'
```

```
{"ok":true,"acknowledged":true}
```

```
curl -XPUT 'http://localhost:9200/magasin/cd/_mapping' -d '{
  "magasin":{
    "_routing":{
      "required":true,
      "path":"genre"
    }
  }
}'
```

On crée un document avec la valeur du routage à **opera**, en utilisant le champ **genre** dans le document.

```
curl -XPOST 'http://localhost:9200/magasin/cd/cd2' -d '{
  "titre": "Le barbier de Séville",
  "compositeur": "Gioacchino ROSSINI",
  "genre": "opera"
}'
```

## 9. Conclusion

Elasticsearch est un moteur de recherche que l'on peut le paramétrer pour améliorer les performances d'indexation et de recherche de documents.

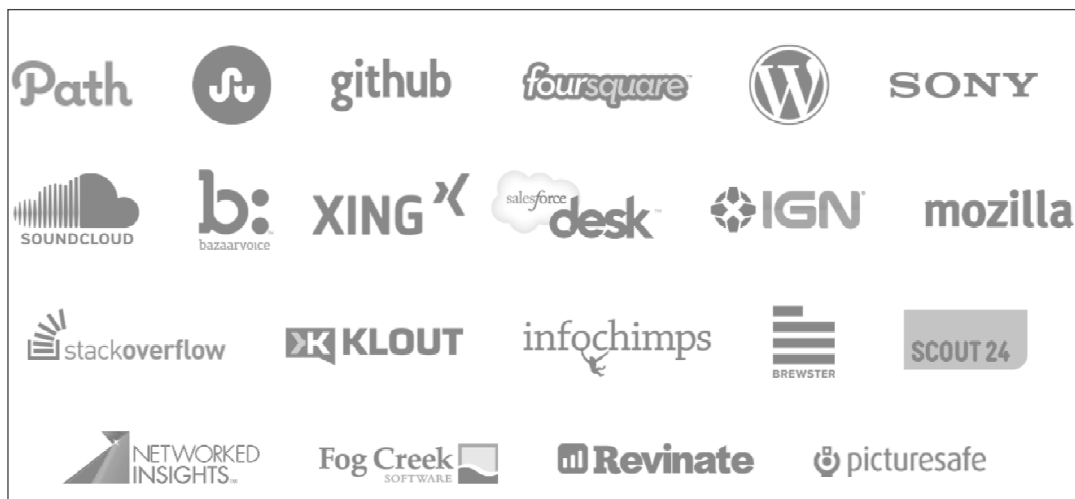
Les documents et les requêtes peuvent être analysés.

On peut faire des recherches par facette ou par percolation.

Le cluster est scalable à l'horizontale et peut avoir un nombre de serveurs important.

Elasticsearch est élastique. Il a des plug-ins que l'on peut ajouter qui offrent des fonctionnalités supplémentaires.

Il est utilisé par des sites et des entreprises dont certains sont connus.



## **Bibliographie :**

### Sites Elasticsearch

- <http://www.elasticsearch.org/guide/>
- Documentation en pdf et en format word  
<http://elasticsearch-users.115913.n3.nabble.com/Documentation-in-printable-format-msWord-and-PDF-td3809547.html>
- <http://www.elasticsearch.cn/blog/2011/02/08/percolator.html>
- <http://www.elasticsearch.org/blog/customizing-your-document-routing>
- Scaling massive elastic search clusters – Rafał Kuć  
<http://www.elasticsearch.org/videos/scaling-massive-elasticsearch-clusters/>

### Livres :

- ElasticSearch Server de Rafal Kuć et Marek Rogozinski (chapitre 1, 2, 6 et 7)
- Mastering ElasticSearch Rafal Kuć et Marek Rogozinski (chapitre 1, 2, 3 et 4)

### Autres sites

- <http://zenika.developpez.com/tutoriels/java/installation-configuration-elasticsearch/>
- Présentation de David Pilato  
<http://fr.slideshare.net/dadoonet/paris-data-geek-elasticsearch>